

LatentSkill: From In-Context Textual Skills to In-Weight Latent Skills for LLM Agents

Aofan Yu, Chenyu Zhou, Tianyi Xu, Zihan Guo, Rong Shan, Zhihui Fu, Jun Wang, Weiwen Liu, Yong Yu, Weinan Zhang, Jianghao Lin

ABSTRACT

Agent systems increasingly use textual skills to encode reusable task procedures, but injecting these skills into the prompt at every step incurs substantial context overhead and exposes skill content as plaintext. We present LatentSkill, a framework that converts textual skills into plug-and-play LoRA adapters through a pretrained hypernetwork. LatentSkill stores skill knowledge in weight space rather than context space, removing per-step skill tokens while preserving modular loading, scaling, and composition. On ALFWorld and Search-QA, LatentSkill outperforms the corresponding in-context skill baseline while using substantially fewer prefill tokens: it improves ALFWorld success by 21.4 and 13.4 points on the seen and unseen splits with 64.1% fewer prefill tokens, and improves Search-QA exact match by 3.0 points with 72.2% lower skill-token overhead. Further analysis shows that generated skill LoRAs form a structured semantic geometry, can be precisely controlled via the LoRA scaling coefficient, and can be composed through parameter-space arithmetic when skill components are aligned. These findings suggest that weight-space skills provide an efficient, modular, and less exposed substrate for extending LLM agents.

About this document

This is a **Reviewed Preprint**: a third-party paper that llmXive has *auto-reviewed* (with its LLM review panel) but *never modified*. The original manuscript follows this cover page **exactly as published** by its authors — llmXive re-typesets nothing and claims no authorship.

Provenance. Ingested by llmXive on 2026-07-02 — scraped from arXiv; via llmXive dashboard, GitHub issue #299; submitted by github-actions[bot].

Source. <https://arxiv.org/abs/2606.06087>

About llmXive. llmXive is an automated platform for open scientific discovery. Its specialist LLM agents advance their own ideas from brainstorm to peer-reviewed paper, and they also review notable third-party papers — drawn from the most-downloaded papers on Hugging Face and from submissions by human contributors. Every submitted paper is reviewed by the LLM panel

and used to seed a new llmXive follow-up study. This paper is one such third-party work: llmXive reviewed it but did not write it. Learn more at <https://github.com/ContextLab/llmXive>.

Automated review. See the accompanying [peer-review-llmxive.pdf](#) for the full reviewer report.

llmXive follow-up. A separate llmXive study building on this work: [PROJ-887-llmxive-follow-up-extending-latentskill](#).

LatentSkill: From In-Context Textual Skills to In-Weight Latent Skills for LLM Agents

Aofan Yu^{1*} Chenyu Zhou^{1*} Tianyi Xu¹ Zihan Guo^{2,3} Rong Shan¹
 Zhihui Fu⁴ Jun Wang^{4†} Weiwen Liu^{1†} Yong Yu¹ Weinan Zhang^{1,3†} Jianghao Lin^{1†}

¹Shanghai Jiao Tong University ²Sun Yat-Sen University

³Shanghai Innovation Institute ⁴OPPO Research Institute

junwang.lu@gmail.com {wwliu, wnzhang, linjianghao}@sjtu.edu.cn

Abstract

Agent systems increasingly use textual skills to encode reusable task procedures, but injecting these skills into the prompt at every step incurs substantial context overhead and exposes skill content as plaintext. We present **LatentSkill**, a framework that converts textual skills into plug-and-play LoRA adapters through a pre-trained hypernetwork. **LatentSkill** stores skill knowledge in weight space rather than context space, removing per-step skill tokens while preserving modular loading, scaling, and composition. On ALFWorld and Search-QA, **LatentSkill** outperforms the corresponding in-context skill baseline while using substantially fewer prefill tokens: it improves ALFWorld success by 21.4 and 13.4 points on the seen and unseen splits with 64.1% fewer prefill tokens, and improves Search-QA exact match by 3.0 points with 72.2% lower skill-token overhead. Further analysis shows that generated skill LoRAs form a structured semantic geometry, can be precisely controlled via the LoRA scaling coefficient, and can be composed through parameter-space arithmetic when skill components are aligned. These findings suggest that weight-space skills provide an efficient, modular, and less exposed substrate for extending LLM agents.¹

1 Introduction

LLM agents increasingly solve complex tasks by interleaving reasoning, action, and feedback from external environments (Yao et al., 2023; Shinn et al., 2023; Zhao et al., 2024). To handle specialized and long-horizon tasks, many systems further rely on external skills: reusable textual procedures that encode task strategies, tool-use patterns, and recovery heuristics (Wang et al., 2023; Xia et al., 2026; Wu et al., 2026; Ouyang et al., 2026; Pan

*Equal contribution.

†Corresponding authors.

¹We provide our code on GitHub: <https://github.com/yuaofan0-oss/LatentSkill>.

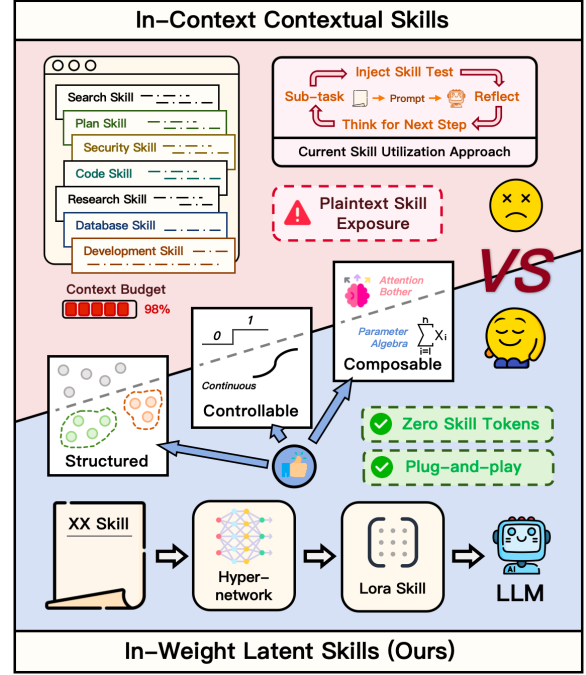


Figure 1: The key advantages of **LatentSkill** over in-context skill: (1) zero skill tokens in prompt with plug-and-play modularity, and (2) a structured, controllable, and composable skill weight space.

et al., 2026; Wang et al., 2026). A common design retrieves relevant skills from a skill library and inserts them into the prompt when the agent selects an action (Cho et al., 2026; Zhang et al., 2026). This design is simple and modular, but it becomes costly as interactions grow longer and skill libraries grow larger. The same skill text may be inserted repeatedly across decision steps, consuming context and increasing prefill cost; long inputs also make it harder for models to use all supplied information robustly (Jiang et al., 2023, 2024; Liu et al., 2024). Moreover, skills kept as readable prompt content can expose proprietary procedures and share the instruction channel with potentially untrusted observations (Greshake et al., 2023; Liu et al., 2025; Wallace et al., 2024; Li et al., 2026b; Guo et al., 2026). Parametric alternatives, such

as agent fine-tuning or curriculum learning, avoid inserting skill text at inference time, but they fuse skills into the model parameters (Chen et al., 2023; Zeng et al., 2024; Lu et al., 2026). As a result, individual skills become difficult to update, remove, or combine. Existing approaches therefore face a trilemma: how to avoid repeated skill text in the prompt, while still allowing skills to be updated modularly and combined during inference.

We introduce **LatentSkill**, a framework that converts textual agent skills into LoRA adapters through hypernetwork-based adapter generation (Ha et al., 2016; Hu et al., 2021; Liu et al., 2026). Instead of delivering skills through the context window, **LatentSkill** represents them in weight space. Given a skill description, a trained hypernetwork generates a skill-specific LoRA adapter in a single forward pass, which is then mounted on the backbone LLM during inference. The original skill text is no longer included in the prompt, reducing both context cost and exposure as readable text. Meanwhile, the generated adapters remain modular: they can be loaded, unloaded, replaced, or scaled without retraining the backbone, aligning with prior work on non-destructive adapter composition and dynamic LoRA combination (Pfeiffer et al., 2021; Huang et al., 2024). They can also be combined when the original skills are decomposed into semantically aligned components.

Beyond efficiency, we show that representing skills as LoRA weights gives them useful structure. Across ALFWorld (Shridhar et al., 2021) and Search-QA (Jin et al., 2025), **LatentSkill** improves over the direct in-context skill baseline while using the same skill descriptions and substantially reducing the prompt overhead caused by skill text. We further find that generated skill LoRAs are **structured**, with skills from different domains forming separable clusters in weight space; **controllable**, since their effect can be adjusted through the LoRA scaling coefficient α ; and **composable**, when skill descriptions are decomposed into aligned components before their LoRAs are combined. Together, these results suggest that latent skill weights are not just compressed prompts, but a representation of procedural knowledge that can be inspected, adjusted, and reused.

Our contributions are threefold. (1) We propose **LatentSkill**, a framework that converts textual agent skills into modular LoRA adapters through a hypernetwork. (2) We show that **LatentSkill** improves over direct in-context skill prompting on

ALFWorld and Search-QA while reducing the context overhead introduced by skill text. (3) We analyze the generated skill weights and show that they exhibit domain-level structure, can be controlled through LoRA scaling, and can be composed when skills are decomposed at the right granularity.

2 Related Work

2.1 LLM Agents and Skill Systems

LLM agents solve complex tasks by interleaving reasoning and action (Yao et al., 2023) and can improve from failures through self-reflection (Shinn et al., 2023). To extend agents beyond the knowledge boundary of a single model, a growing body of work has explored injecting external experiential knowledge at decision time (Zhou et al., 2026). Early approaches store raw trajectories or reflective summaries in external memory banks and retrieve them into the context during inference (Zhao et al., 2024; Chhikara et al., 2025). Subsequent work converged on skills as the core abstraction: Wang et al. (2023) introduced an ever-growing library of executable skills for embodied agents; Xia et al. (2026) distill trajectories into a hierarchical SkillBank that co-evolves with the agent’s policy through recursive reinforcement learning; and Wu et al. (2026) refine interaction experience into reusable strategic principles. On the industry side, Anthropic’s Agent Skills specification has been adopted by mainstream harnesses including Claude Code, Cursor, and Gemini CLI (Li et al., 2026a; Yang et al., 2025b). All of the above deliver skills as natural-language text injected into the context window. Lu et al. (2026) take a different direction by internalizing skills into model parameters through a training-time curriculum that progressively withdraws skill context, enabling zero-shot execution at inference. Their skills, however, are irreversibly fused into the backbone, forfeiting modular flexibility. In contrast, **LatentSkill** encodes skills as LoRA modules that are independent of the backbone, simultaneously achieving zero context overhead and plug-and-play flexibility.

2.2 Hypernetworks for LoRA Generation

Hypernetworks generate the weights of a target network through a separate learned network (Ha et al., 2016) and have recently been applied to produce LoRA adapters (Hu et al., 2021) for LLMs in a single forward pass, bypassing the iterative overhead of per-task fine-tuning. Text-to-LoRA (Charakorn

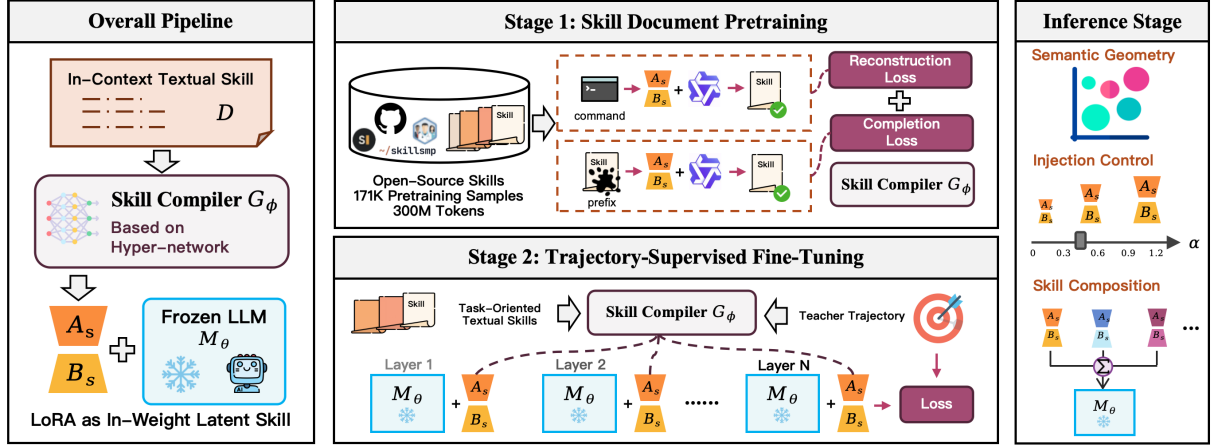


Figure 2: Overview of **LatentSkill**. **Left**: textual skills are transformed into in-weight latent skills through hypernetwork-based LoRA generation. **Middle**: the skill compiler is trained by skill document pretraining and trajectory-supervised fine-tuning. **Right**: the resulting latent skills support structured semantic geometry, controllable injection strength, and composable parameter-space arithmetic at inference time.

et al., 2025) employs small MLPs to independently generate weight segments for each layer and concatenate them into a complete adapter, but this segment-wise strategy fails to capture global dependencies across layers. Generative Adapter (Chen et al., 2024) utilizes the hidden states of the backbone LLM to produce LoRA weights, yet its parameter cost restricts coverage to only a subset of target modules. ICAE (Ge et al., 2024) compresses contexts into a fixed set of compact tokens before generating LoRA, but is constrained by the resulting information bottleneck. SHINE (Liu et al., 2026) extracts memory states at every layer and enables bidirectional cross-layer information flow through an attention mechanism, but does not analyze the intrinsic structure of the generated LoRA weight space. Doc-to-LoRA (Charakorn et al., 2026) generates adapters through a Perceiver-style encoder, but likewise focuses on context internalization. In contrast, **LatentSkill** targets the agent skill encoding setting and is the first to systematically reveal that the weight space of hypernetwork-generated LoRA exhibits structuredness, controllability, and composability.

3 Method

LatentSkill converts a textual skill into a LoRA adapter that can be mounted on a frozen LLM. The skill text is consumed by a skill compiler, while inference uses the generated adapter instead of inserting the skill document into the prompt. We describe it in four parts: latent skill definition, document-level pretraining, trajectory-supervised fine-tuning, and inference-time control and composition.

3.1 Latent Skill Definition

Let M_θ denote a backbone LLM with frozen parameters θ , and let s be a textual skill document. At decision step t , the agent observes a history h_t and produces an output y_t . In-context skill prompting directly conditions the model on the skill text. **LatentSkill** instead replaces textual conditioning with parameter conditioning.

A skill compiler G_ϕ maps the skill document to a set of LoRA updates:

$$\Delta_s = G_\phi(s), \quad (1)$$

where Δ_s denotes the latent skill induced by s . With Δ_s mounted, the model predicts from the task history alone:

$$p_{\theta, \phi}(y_t | h_t, s) = p_{\theta \oplus \alpha \Delta_s}(y_t | h_t), \quad (2)$$

where $\oplus$ denotes LoRA-based parameter augmentation and α controls the injection strength.

For each target module $m \in \mathcal{M}$, the generated update follows the standard LoRA form $\Delta W_s^{(m)} = B_s^{(m)} A_s^{(m)}$. Mounting a latent skill adds a scaled low-rank update to the frozen weight:

$$W' = W + \frac{\alpha}{r} B_s A_s, \quad (3)$$

where r is the LoRA rank. A complete latent skill is the collection of generated updates $\Delta_s = \{\Delta W_s^{(m)}\}_{m \in \mathcal{M}}$ over the selected target modules.

3.2 Skill Document Pretraining

First, we pretrain the skill compiler on a corpus of textual skill documents, denoted by $\mathcal{D}_{pre} =$

$\{s_i\}_{i=1}^N$. The goal is to initialize G_ϕ to map procedural text into usable adapter weights while keeping the backbone LLM frozen. Given a skill document s , we randomly instantiate one of two document-level pretraining tasks. In the reconstruction task, the compiler reads the complete skill document s , and the adapted backbone receives a reconstruction instruction as input and is trained to reproduce the original document s . In the completion task, we construct a truncated prefix $\tilde{s}$ by randomly removing the latter part of the document; the compiler reads $\tilde{s}$, and the adapted backbone is trained to complete the full skill document.

For each skill, we construct document-level supervision instances $(s_i^{\text{src}}, q_i, z_i)$, where s_i^{src} is the text provided to the compiler, q_i is the prompt given to the adapted backbone, and z_i is the target output. Let $\Delta_i^{\text{pre}} = G_\phi(s_i^{\text{src}})$. The pretraining objective is

$$\mathcal{L}_{\text{pre}} = - \sum_{i,j} \log p_{\theta \oplus \alpha \Delta_i^{\text{pre}}}(z_{i,j} \mid q_i, z_{i,<j}), \quad (4)$$

where the summation ranges over all document-level supervision instances and target tokens.

Only the compiler parameters ϕ are updated. Since the skill document is provided to G_ϕ rather than directly to the adapted backbone, information useful for predicting z_i must be mediated through the generated adapter.

3.3 Trajectory-Supervised Fine-Tuning

After pretraining, we fine-tune the skill compiler with teacher agent trajectories. Let $\mathcal{D}_{\text{sft}} = \{(s_i, \tau_i)\}_{i=1}^M$ denote the supervised dataset. Each example pairs a skill document s_i with a teacher trajectory $\tau_i = \{(h_{i,t}, y_{i,t}^*)\}_{t=1}^{T_i}$, where $h_{i,t}$ is the agent history at step t and $y_{i,t}^*$ is the teacher output.

For each pair (s_i, τ_i) , the compiler generates one latent skill, denoted by $\Delta_i^{\text{sft}} = G_\phi(s_i)$. The same adapter is mounted throughout the entire trajectory. The fine-tuning objective is

$$\mathcal{L}_{\text{sft}} = - \sum_{i,t,j} \log p_{\theta \oplus \alpha \Delta_i^{\text{sft}}}(y_{i,t,j}^* \mid h_{i,t}, y_{i,t,<j}^*), \quad (5)$$

where the summation ranges over all trajectories, decision steps, and target tokens.

The backbone remains frozen, and only ϕ is updated. Since Δ_i^{sft} is generated solely from the skill document s_i and shared across all decision steps in τ_i , the objective encourages the adapter to capture skill-level, trajectory-consistent policy information rather than per-step adaptations. This aligns the

compiler to produce latent skills whose effects remain stable across multi-step interaction.

3.4 Inference-Time Skill Control and Composition

At inference time, skill compilation is separated from agent execution. Given a skill library $\mathcal{S} = \{s_1, \dots, s_K\}$, each skill can be compiled once and stored in an adapter cache $\mathcal{C}[k] = G_\phi(s_k)$. After compilation, the skill is not included in the prompt.

For a task instance, a skill selector chooses one or more relevant skills. If a single skill s_k is selected, its cached adapter $\mathcal{C}[k]$ is mounted on the backbone with injection coefficient α_k . The agent then predicts each step from the current history h_t using the adapted model. Setting $\alpha_k = 0$ recovers the frozen backbone, while larger values increase the influence of the latent skill.

When multiple skills are selected, **LatentSkill** composes their adapters in weight space:

$$\Delta_{\mathcal{K}} = \sum_{k \in \mathcal{K}} \alpha_k \mathcal{C}[k], \quad (6)$$

where $\mathcal{K}$ is the selected skill set. The composed adapter is then mounted on the LLM for inference.

For skills with shared subcomponents, direct adapter addition may over-amplify common behavior. **LatentSkill** therefore also supports component-level composition. Specifically, a skill can be decomposed into semantic components $s_k = \{c_{k,1}, \dots, c_{k,L_k}\}$, each component can be compiled independently as $\Delta_{k,\ell} = G_\phi(c_{k,\ell})$, and the final adapter can be formed by adding retained shared and skill-specific components, e.g., $\Delta_{\text{comp}} = \sum_{c \in \mathcal{U}} \gamma_c G_\phi(c)$, where $\mathcal{U}$ is the selected component set and γ_c is an optional component-level injection coefficient.

4 Experiments

4.1 Experiment Setup

Benchmarks. We evaluate **LatentSkill** on two agent benchmarks. **ALFWorld** (Shridhar et al., 2021) is a text-based interactive environment aligned with the ALFRED embodied AI benchmark, comprising six categories of household tasks: Pick and Place (Pick), Look at Obj in Light (Look), Pick Clean then Place in Recep (Clean), Pick Heat then Place in Recep (Heat), Pick Cool then Place in Recep (Cool), and Pick Two Obj and Place (Pick2). **Search-QA** follows the evaluation protocol of Jin et al. (2025) and covers seven search-augmented QA datasets, including three single-hop

benchmarks (NQ (Kwiatkowski et al., 2019), TriviaQA (Joshi et al., 2017), PopQA (Mallen et al., 2023)) and four multi-hop benchmarks (HotpotQA (Yang et al., 2018), 2WikiMultihopQA (Ho et al., 2020), MuSiQue (Trivedi et al., 2022), Bamboogle (Press et al., 2023)). Training data are drawn from NQ and HotpotQA, and the remaining five datasets serve as out-of-domain evaluation sets.

Baselines. For ALFWorld, we compare against Vanilla, Few-shot, Reflexion (Shinn et al., 2023), AdaPlanner (Sun et al., 2023), and In-context Skill. For Search-QA, we compare against Vanilla, CoT (Wei et al., 2023), Few-shot, R1-Instruct (Guo et al., 2025), RAG (Lewis et al., 2020), and In-context Skill. Among them, In-context Skill uses the same skill as **LatentSkill** but places it in the prompt rather than encoding it as LoRA weights.

Implementation Details. We use Qwen3-8B (Yang et al., 2025a) as the frozen backbone LLM. The skill compiler is implemented as a Transformer-based hypernetwork that maps each textual skill document to a set of LoRA updates. Unless otherwise specified, we use the same LoRA configuration across experiments. In the pretraining stage, we train the compiler on approximately 171K deduplicated skill documents crawled from GitHub, totaling roughly 300M tokens. In the SFT stage, we use the teacher trajectories and skill library released by Xia et al. (2026), mixing ALFWorld and Search-QA trajectories into a single training set and using 5 ALFWorld skills and 3 Search-QA skills. For Search-QA, passage retrieval is performed using E5 (Wang et al., 2024). Complete training hyperparameters and skill-to-task matching rules are provided in Appendix A and Appendix B.

4.2 Main Results

Method Performance. Tables 1 and 2 show that **LatentSkill** achieves the best average performance on both benchmarks. On ALFWorld, **LatentSkill** reaches 74.3% and 69.4% average success on both the seen and unseen splits, improving over In-Context Skill by 21.4 and 13.4 points. On Search-QA, it achieves the highest average EM score of 35.6. The gains are especially pronounced on multi-step tasks: on unseen Pick2, **LatentSkill** reaches 70.6%, surpassing the second-best method by 41.2 points, and it also obtains the best scores on HotpotQA and MuSiQue.

Token Efficiency. On ALFWorld, **LatentSkill** reduces prefill overhead by 64.1% relative to In-Context Skill, while improving average success by 21.4 and 13.4 points on the seen and unseen splits, respectively. On Search-QA, it reduces context overhead by 72.2% and improves average EM by 3.0 points. These results suggest that moving skill knowledge from prompt to LoRA weight space provides a dual benefit: it lowers inference-time token cost while preserving, and often improving, the effectiveness of behavior.

Beyond cost, **LatentSkill** also shortens interaction trajectories. On the seen split, it achieves the fewest average steps per episode, reducing the trajectory length from 35.0 steps for the Vanilla backbone to 28.4 steps. This indicates that latent skill weights can help the agent reach successful outcomes with fewer environment interactions, rather than merely improving final success rates.

Tables 1 and 2 show that **LatentSkill** improves both performance and token efficiency over explicit text injection. We next ask whether the resulting skill weight space has structure that can be analyzed and used. We find that **LatentSkill** exhibits three properties: *structured*, *controllable*, and *composability*.

4.3 Structured: Semantic Geometry of the LoRA Weight Space

To examine whether **LatentSkill** form a meaningful latent geometry, we apply Multidimensional Scaling (MDS) to the LoRA weights of the 8 in-domain skills at both the pretrain and SFT stages. We also evaluate out-of-distribution (OOD) skill texts collected from public GitHub repositories, covering Code (18 skills), Finance (13 skills), and Writing (11 skills). OOD source details are provided in Appendix F.

As shown in the left panel of Figure 3, the in-domain skill LoRAs form clear domain-level clusters: the 5 ALFWorld skills and 3 Search skills are separated in weight space, with an inter-cluster distance of 0.0887 and higher within-domain than cross-domain similarity (0.982 vs. 0.910). After SFT, the inter-cluster distance decreases to 0.0704, a 20.6% reduction, while both within-domain and cross-domain similarities increase. This suggests that SFT introduces shared agent-level behavioral patterns while preserving skill-specific structure.

The right panel further shows that this semantic organization generalizes beyond the training domains. OOD skills from Code, Finance, and Writ-

Table 1: Performance on **ALFWorld** in success rate %. Results are reported on both *seen* and *unseen* splits with per-task breakdown. *Step* denotes the average number of steps taken per episode. *Prefill* and *Decode* report average token counts (k) per step. The best and second-best results are highlighted in **bold** and underline, respectively.

Method	ALFWorld Task						Avg↑	Step↓	Cost↓	
	Pick	Look	Clean	Heat	Cool	Pick2			Prefill	Decode
<i>Seen split</i>										
Vanilla	82.9	46.2	18.5	37.5	32.0	29.2	43.6	35.0	0.44	0.55
Few-Shot	82.9	46.2	44.4	68.8	36.0	12.5	50.0	31.3	2.04	0.54
Reflexion	77.1	53.9	33.3	31.3	<u>56.0</u>	12.5	46.4	33.3	0.55	0.66
AdaPlanner	<u>91.4</u>	30.8	33.3	37.5	28.0	<u>33.3</u>	47.1	35.9	0.54	0.84
In-Context Skill	<u>85.7</u>	<u>69.2</u>	70.4	31.3	12.0	<u>33.3</u>	<u>52.9</u>	<u>30.8</u>	1.21	<u>0.50</u>
LatentSkill	97.1	92.3	<u>63.0</u>	<u>43.8</u>	64.0	75.0	74.3	28.4	0.44	0.34
<i>Unseen split</i>										
Vanilla	54.2	55.6	41.9	<u>47.8</u>	57.1	23.5	47.0	34.9	0.44	0.62
Few-Shot	66.7	50.0	61.3	56.5	<u>76.2</u>	0.00	54.5	28.5	2.04	0.50
Reflexion	50.0	<u>61.1</u>	58.1	43.5	66.7	0.00	48.5	32.8	0.57	0.82
AdaPlanner	<u>83.3</u>	<u>50.0</u>	54.8	39.1	52.4	<u>29.4</u>	53.0	34.3	0.54	0.75
In-Context Skill	70.8	<u>61.1</u>	74.2	43.5	47.6	23.5	<u>56.0</u>	<u>29.7</u>	1.23	0.61
LatentSkill	91.7	66.7	<u>64.5</u>	43.5	81.0	70.6	69.4	31.4	0.44	<u>0.51</u>

Table 2: Performance on **Search-QA** in exact match %. $\dagger$ and $\star$ indicate in-domain and out-of-domain datasets, respectively. *Cost* denotes average token counts (k) per step. The best and second-best results are highlighted in **bold** and underline, respectively.

Method	Single-Hop QA			Multi-Hop QA				Avg $\uparrow$	Cost $\downarrow$
	NQ $\dagger$	Triv $\star$	Pop $\star$	Hotp $\dagger$	2WK $\star$	MuS $\star$	Bam $\star$		
Vanilla	25.2	50.6	35.2	26.2	26.8	4.20	28.8	28.1	0.24
CoT	19.2	50.8	20.0	22.4	26.2	5.00	<u>37.6</u>	24.5	0.09
Few-Shot	34.6	<u>57.6</u>	39.4	30.0	25.8	6.00	17.6	31.7	0.94
R1-Instruct	27.0	55.0	31.6	26.6	<u>33.6</u>	5.80	34.4	30.1	<u>0.24</u>
RAG	39.0	64.0	45.0	32.4	21.2	6.80	27.2	34.4	0.89
In-Context Skill	27.2	56.4	33.0	30.2	39.8	<u>7.60</u>	38.4	32.6	1.10
LatentSkill	<u>36.2</u>	<u>57.6</u>	<u>41.0</u>	39.6	32.0	9.80	25.6	35.6	0.31

ing form separated clusters, with within-domain similarities of 0.783, 0.9664, and 0.9681, respectively, each exceeding the corresponding cross-domain similarities. Since these skills come from diverse repositories and writing styles without domain-specific supervision, the clustering suggests that the compiler maps procedural text into a semantically organized weight space rather than merely overfitting to the in-domain skills.

Given this geometric structure, a natural question arises: can we leverage this structure to actively control the injection strength of a skill?

4.4 Controllable: Precise Modulation of Skill Injection Strength

We next examine whether **LatentSkill** support continuous strength control, rather than the binary choice available to in-context skill. We introduce an injection coefficient α that linearly scales the hypernetwork output ΔW , and sweep

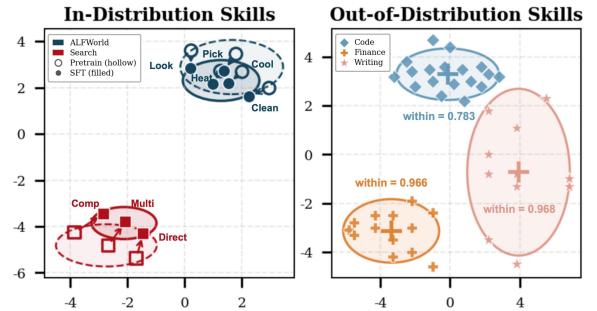


Figure 3: MDS visualization of LoRA weights. **Left:** in-domain ALFWorld and Search skills; **Right:** OOD Code, Finance, and Writing skills; “+” marks each cluster centroid, and *within* reports mean intra-cluster cosine similarity. Axes are scaled by $\times 10^{-2}$.

$\alpha \in \{0, 0.1, 0.2, 0.3, 0.5, 0.6, 0.8, 1.0, 1.2\}$ on the ALFWorld seen and unseen splits. Here, $\alpha=0$ corresponds to the frozen backbone without LoRA. Full results are provided in Table 11.

As shown in Figure 4, task performance follows an inverted-U curve. On the seen split, average success rises from 43.57% at $\alpha=0$ to 74.29% at $\alpha=0.6$, but drops to 22.86% at $\alpha=1.2$. The unseen split shows the same pattern, peaking at 70.90% when $\alpha=0.5$ and falling to 8.21% at $\alpha=1.2$. Moreover, four of the six tasks share the same optimal α across seen and unseen splits, while the remaining two differ by only one grid point. These results suggest that generated skill LoRAs have a stable effective injection range: moderate scaling strengthens skill behavior, whereas excessive scaling disrupts the backbone. However, the optimal α varies across tasks, and using a single global value can be suboptimal; for example, applying $\alpha=0.6$ to all unseen tasks loses 21.74, 17.65, and 12.90 points on Heat, Pick2, and Clean relative to their task-specific optima, motivating adaptive α selection.

Figure 4 further suggests that tasks with weaker backbone baselines often require stronger skill injection. Pick and Pick2 share the same skill, but Pick2 has a lower unseen-split backbone baseline than Pick, 23.53% compared with 54.17%, and requires a larger optimal α , 0.8 compared with 0.6. With this stronger injection, Pick2 reaches 88.24% on the unseen split, close to the peak performance of Pick on the same split at 91.67%. A similar pattern appears across different skills: Clean starts from a lower baseline than Cool, requires a higher optimal α , and obtains a larger peak gain. Heat is an exception, suggesting that optimal injection strength also depends on the structure of the required action sequence.

The α -scaling analysis demonstrates that the injection strength of individual skills can be precisely modulated. A further question naturally follows: can different skills be directly combined within this structured space?

4.5 Composable: Skill Arithmetic in Parameter Space

We next examine whether **LatentSkill** can be composed in parameter space. We define successful composition as preserving the target skill capability while adding the complementary capability of an auxiliary skill without interference. We use *Look* as the target skill and *Pick* as the auxiliary skill, since their task-specific components are non-overlapping and complementary in ALFWorld. We evaluate five configurations on all 31 Look episodes: Look-Only, Pick-Only, Direct Merging

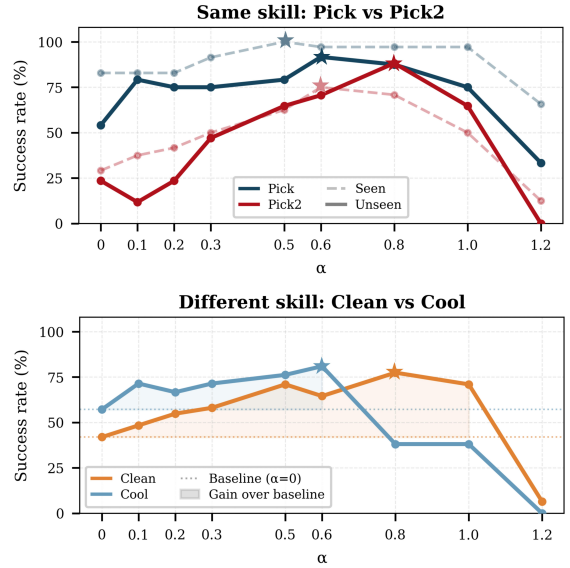


Figure 4: Scale-performance curves on ALFWorld under varying LoRA injection coefficient α . **Top:** Pick vs. Pick2, the same skill but differing in difficulty. Stars mark the per-task optimal α . **Bottom:** Clean vs. Cool on the unseen split, using different skills. Shaded regions indicate the performance gain over the $\alpha=0$ baseline.

of complete skill LoRAs, Text Merging by concatenating skill texts before compilation, and Component Merging by separately compiling aligned components and combining their LoRAs.

As shown in Table 3, Component Merging achieves the best performance on both splits, reaching 84.6% on seen episodes and 77.8% on unseen episodes. Compared with Look-Only, it adds 3 successful episodes on the seen split and 2 on the unseen split, while losing none of Look-Only’s original successes. This satisfies both requirements of our composability definition: preserving the target capability and introducing complementary behavior. By contrast, Direct Merging and Text Merging fail to improve over Look-Only on the unseen split and lose some of its original successes, suggesting that naive weight averaging and text concatenation both introduce interference.

Per-episode analysis in Appendix H suggests that Component Merging succeeds because it aligns the granularity of text decomposition with the granularity of weight addition. The Pick-specific component contributes systematic search behavior, while shared general and mistake-avoidance components are included only once, avoiding redundant amplification. Direct Merging instead double-counts shared components at the weight level, while Text Merging creates an out-

Table 3: Skill composition results on all 31 Look task episodes under five skill composition configurations. The best result per split is highlighted in **bold**.

Method	Seen		Unseen	
	Ep.	%	Ep.	%
Look-Only	8/13	61.5	13/18	72.2
Pick-Only	8/13	61.5	11/18	61.1
Direct Merging	9/13	69.2	11/18	61.1
Text Merging	8/13	61.5	11/18	61.1
Component Merging	11/13	84.6	14/18	77.8

of-distribution combined skill document. These results suggest that skill composition in LoRA space requires semantically aligned components rather than direct merging of whole skills. These results suggest a general principle: skill composition in LoRA space requires semantic alignment between text decomposition and weight addition. Direct Merging over-amplifies shared components at the weight level, while Text Merging creates an out-of-distribution combined skill document at the text level. Component Merging avoids both failure modes by retaining shared components once and adding task-specific components on demand, enabling complementary skills to coexist with less interference.

5 Sensitivity and Security

We further evaluate robustness when the skill text is perturbed or the prompt is attacked. For sensitivity analysis, we apply four perturbations to the skill text: **Paraphrase**, which rewrites the skill with equivalent meaning; **Plaintext**, which removes Markdown formatting; **Reorder**, which shuffles bullet order within each section; and **Noise**, which injects irrelevant but fluent descriptions. For security analysis, we consider two prompt-level attacks: **Hijack**, which appends a malicious system-override instruction, and **Extract**, which asks the model to reproduce the skill content. Implementation details and full per-task results are provided in Appendix I.

As shown in Table 4, **LatentSkill** maintains a consistent advantage over In-context Skill under all perturbations. On ALFWorld, the margin ranges from 17.2 to 24.3 points, comparable to the 21.4 point margin under the base condition. On Search-QA, the latent-skill approach also remains ahead under every perturbation. Notably, removing Markdown formatting causes no degradation on

Table 4: Average performance under skill text perturbations and adversarial attacks. ALFWorld reports success rate (%) and Search-QA reports exact match (%).

Method	ALFWorld		Search-QA	
	In-context	Latent	In-context	Latent
Base	52.9	74.3	32.6	35.6
Paraphrase	50.7	67.9	33.2	34.0
Plaintext	50.0	74.3	32.4	34.4
Reorder	50.7	69.3	32.4	33.6
Noise	47.9	71.4	31.7	33.7
Hijack	8.57	38.6	23.5	34.0
Extract	48.6	70.0	21.3	29.3

ALFWorld, suggesting that the generated LoRA captures skill semantics rather than relying on surface formatting. Across the four perturbations, the average ALFWorld success rate is 70.7%, only 3.6 points below the base performance.

The same pattern holds under prompt-level attacks. Under Hijack, In-context Skill drops from 52.9% to 8.57% on ALFWorld, while the weight-space variant retains 38.6%; on Search-QA, it drops by only 1.6 points. Under Extract, In-context Skill is vulnerable because the skill text is directly present in the prompt, whereas weight-space storage reduces direct plaintext exposure. These results suggest that moving skills from prompt space to LoRA weights improves not only efficiency, but also robustness to prompt perturbations and prompt-level attacks.

6 Conclusion

LatentSkill converts textual agent skills into modular LoRA adapters through a pretrained hypernetwork, moving reusable procedural knowledge from context space into weight space. Across ALFWorld and Search-QA, this design improves over direct in-context skill prompting while substantially reducing the repeated prefill overhead introduced by skill text. Beyond efficiency, our analyses show that the generated skill LoRAs form a structured semantic geometry, can be controlled through the injection coefficient, and can be composed in parameter space when skill components are properly aligned. These results suggest that latent skill weights offer a practical substrate for building LLM agents whose skills are efficient, modular, controllable, and less directly exposed as plaintext prompts.

Limitations

This work evaluates **LatentSkill** on two agent benchmarks, ALFWorld and Search-QA, which cover embodied interaction and search-augmented question answering. While these benchmarks span distinct task modalities, they do not exhaust the full diversity of agent deployment scenarios. Evaluating on additional settings such as web browsing, software engineering, and multi-agent collaboration would further validate the generality and scalability of the framework.

In addition, all experiments use Qwen3-8B as the frozen backbone LLM with a fixed LoRA configuration. The behavior of the skill compiler and the properties of the generated latent skills may vary with different model families, model scales, or adapter configurations. Exploring these axes remains a valuable direction for future work.

References

- Rujikorn Charakorn, Edoardo Cetin, Yujin Tang, and Robert Tjarko Lange. 2025. [Text-to-lora: Instant transformer adaption](#). *Preprint*, arXiv:2506.06105.
- Rujikorn Charakorn, Edoardo Cetin, Shinnosuke Uesaka, and Robert Tjarko Lange. 2026. [Doc-to-lora: Learning to instantly internalize contexts](#). *Preprint*, arXiv:2602.15902.
- Baian Chen, Chang Shu, Ehsan Shareghi, Nigel Collier, Karthik Narasimhan, and Shunyu Yao. 2023. [Fire-act: Toward language agent fine-tuning](#). *Preprint*, arXiv:2310.05915.
- Tong Chen, Hao Fang, Patrick Xia, Xiaodong Liu, Benjamin Van Durme, Luke Zettlemoyer, Jianfeng Gao, and Hao Cheng. 2024. [Generative adapter: Contextualizing language models in parameters with a single forward pass](#). *Preprint*, arXiv:2411.05877.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. [Mem0: Building production-ready ai agents with scalable long-term memory](#). *Preprint*, arXiv:2504.19413.
- Hongcheol Cho, Ryangkyung Kang, and Youngeun Kim. 2026. [Skillret: A large-scale benchmark for skill retrieval in llm agents](#). *Preprint*, arXiv:2605.05726.
- Tao Ge, Jing Hu, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. 2024. [In-context autoencoder for context compression in a large language model](#). *Preprint*, arXiv:2307.06945.
- Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. [Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injection](#). In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security, AISec ’23*, pages 79–90, New York, NY, USA. Association for Computing Machinery.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, and 175 others. 2025. [Deepseek-r1 incentivizes reasoning in llms through reinforcement learning](#). *Nature*, 645(8081):633–638.
- Zihan Guo, Zhiyu Chen, Xiaohang Nie, Jianghao Lin, Yuanjian Zhou, and Weinan Zhang. 2026. [Skill-probe: Security auditing for emerging agent skill marketplaces via multi-agent collaboration](#). *Preprint*, arXiv:2603.21019.
- David Ha, Andrew Dai, and Quoc V. Le. 2016. [Hyper-networks](#). *Preprint*, arXiv:1609.09106.
- Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. 2020. [Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps](#). In *Proceedings of the 28th International Conference on Computational Linguistics*, pages 6609–6625, Barcelona, Spain (Online). International Committee on Computational Linguistics.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. [Lora: Low-rank adaptation of large language models](#). *Preprint*, arXiv:2106.09685.
- Chengsong Huang, Qian Liu, Bill Yuchen Lin, Tianyu Pang, Chao Du, and Min Lin. 2024. [Lorahub: Efficient cross-task generalization via dynamic lora composition](#). *Preprint*, arXiv:2307.13269.
- Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. [LLMLingua: Compressing prompts for accelerated inference of large language models](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 13358–13376, Singapore. Association for Computational Linguistics.
- Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2024. [LongLLMLingua: Accelerating LLMs in long context scenarios via prompt compression](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1658–1677, Bangkok, Thailand. Association for Computational Linguistics.
- Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Serkan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. 2025. [Search-r1: Training llms to reason and leverage search engines with reinforcement learning](#). *Preprint*, arXiv:2503.09516.
- Mandar Joshi, Eunsol Choi, Daniel Weld, and Luke Zettlemoyer. 2017. [TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension](#). In *Proceedings of the 55th Annual Meeting of*

- the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1601–1611, Vancouver, Canada. Association for Computational Linguistics.
- Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. 2019. [Natural questions: A benchmark for question answering research](#). *Transactions of the Association for Computational Linguistics*, 7:452–466.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems, NIPS ’20*, Red Hook, NY, USA. Curran Associates Inc.
- Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, Shuyi Wang, Binxu Li, Qunhong Zeng, Di Wang, Xuandong Zhao, Yuanli Wang, Roey Ben Chaim, Zonglin Di, Yipeng Gao, and 22 others. 2026a. [Skillsbench: Benchmarking how well agent skills work across diverse tasks](#). *Preprint*, arXiv:2602.12670.
- Zhiyuan Li, Jingzheng Wu, Xiang Ling, Xing Cui, and Tianyue Luo. 2026b. [Towards secure agent skills: Architecture, threat taxonomy, and security analysis](#). *Preprint*, arXiv:2604.02837.
- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. [Lost in the middle: How language models use long contexts](#). *Transactions of the Association for Computational Linguistics*, 12:157–173.
- Yewei Liu, Xiyuan Wang, Yansheng Mao, Yoav Gelber, Hagai Maron, and Muhan Zhang. 2026. [Shine: A scalable in-context hypernetwork for mapping context to lora in a single pass](#). *Preprint*, arXiv:2602.06358.
- Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Zihao Wang, Xiaofeng Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, Leo Yu Zhang, and Yang Liu. 2025. [Prompt injection attack against llm-integrated applications](#). *Preprint*, arXiv:2306.05499.
- Zhengxi Lu, Zhiyuan Yao, Jinyang Wu, Chengcheng Han, Qi Gu, Xunliang Cai, Weiming Lu, Jun Xiao, Yueting Zhuang, and Yongliang Shen. 2026. [Skill0: In-context agentic reinforcement learning for skill internalization](#). *Preprint*, arXiv:2604.02268.
- Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. [When not to trust language models: Investigating effectiveness of parametric and non-parametric memories](#). In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9802–9822, Toronto, Canada. Association for Computational Linguistics.
- Siru Ouyang, Jun Yan, Yanfei Chen, Rujun Han, Zifeng Wang, Bhavana Dalvi Mishra, Rui Meng, Chun-Liang Li, Yizhu Jiao, Kaiwen Zha, Maohao Shen, Vishy Tirumalashetty, George Lee, Jiawei Han, Tomas Pfister, and Chen-Yu Lee. 2026. [Skillos: Learning skill curation for self-evolving agents](#). *Preprint*, arXiv:2605.06614.
- Shuai Pan, Yixiang Liu, Jiaye Gao, Te Gao, Weiwen Liu, Jianghao Lin, Zhihui Fu, Jun Wang, Weinan Zhang, and Yong Yu. 2026. [Skillmas: Skill co-evolution with llm-based multi-agent system](#). *Preprint*, arXiv:2605.09341.
- Jonas Pfeiffer, Aishwarya Kamath, Andreas Rücklé, Kyunghyun Cho, and Iryna Gurevych. 2021. [AdapterFusion: Non-destructive task composition for transfer learning](#). In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pages 487–503, Online. Association for Computational Linguistics.
- Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah Smith, and Mike Lewis. 2023. [Measuring and narrowing the compositionality gap in language models](#). In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 5687–5711, Singapore. Association for Computational Linguistics.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: language agents with verbal reinforcement learning. In *Proceedings of the 37th International Conference on Neural Information Processing Systems, NIPS ’23*, Red Hook, NY, USA. Curran Associates Inc.
- Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. 2021. [Alfworld: Aligning text and embodied environments for interactive learning](#). *Preprint*, arXiv:2010.03768.
- Haotian Sun, Yuchen Zhuang, Ling kai Kong, Bo Dai, and Chao Zhang. 2023. [Adaplaner: Adaptive planning from feedback with language models](#). *Preprint*, arXiv:2305.16653.
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. [MuSiQue: Multi-hop questions via single-hop question composition](#). *Transactions of the Association for Computational Linguistics*, 10:539–554.
- Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. 2024. [The instruction hierarchy: Training llms to prioritize privileged instructions](#). *Preprint*, arXiv:2404.13208.

- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*.
- Jingxing Wang, Chenyu Zhou, Zhihui Fu, Jun Wang, Weiwen Liu, Weinan Zhang, and Jianghao Lin. 2026. *Skills on the fly: Test-time adaptive skill synthesis for llm agents*. *Preprint*, arXiv:2605.16986.
- Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. 2024. *Text embeddings by weakly-supervised contrastive pre-training*. *Preprint*, arXiv:2212.03533.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2023. *Chain-of-thought prompting elicits reasoning in large language models*. *Preprint*, arXiv:2201.11903.
- Rong Wu, Xiaoman Wang, Jianbiao Mei, Pinlong Cai, Daocheng Fu, Cheng Yang, Licheng Wen, Xueming Yang, Yufan Shen, Yuxin Wang, and Botian Shi. 2026. *Evolver: Self-evolving llm agents through an experience-driven lifecycle*. *Preprint*, arXiv:2510.16079.
- Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. 2026. *Skillrl: Evolving agents via recursive skill-augmented reinforcement learning*. *Preprint*, arXiv:2602.08234.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, and 41 others. 2025a. *Qwen3 technical report*. *Preprint*, arXiv:2505.09388.
- Yingxuan Yang, Huacan Chai, Yuanyi Song, Siyuan Qi, Muning Wen, Ning Li, Junwei Liao, Haoyi Hu, Jianghao Lin, Gaowei Chang, Weiwen Liu, Ying Wen, Yong Yu, and Weinan Zhang. 2025b. *A survey of ai agent protocols*. *Preprint*, arXiv:2504.16736.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. *HotpotQA: A dataset for diverse, explainable multi-hop question answering*. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, Brussels, Belgium. Association for Computational Linguistics.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. *React: Synergizing reasoning and acting in language models*. *Preprint*, arXiv:2210.03629.
- Aohan Zeng, Mingdao Liu, Rui Lu, Bowen Wang, Xiao Liu, Yuxiao Dong, and Jie Tang. 2024. *AgentTuning: Enabling generalized agent abilities for LLMs*. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 3053–3077, Bangkok, Thailand. Association for Computational Linguistics.
- Kangning Zhang, Shuai Shao, Qingyao Li, Jianghao Lin, Lingyue Fu, Shijian Wang, Wenxiang Jiao, Yuan Lu, Weiwen Liu, Weinan Zhang, and Yong Yu. 2026. *Mmskills: Towards multimodal skills for general visual agents*. *Preprint*, arXiv:2605.13527.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. *Expel: Llm agents are experiential learners*. In *Proceedings of the Thirty-Eighth AAAI Conference on Artificial Intelligence and Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence and Fourteenth Symposium on Educational Advances in Artificial Intelligence, AAAI’24/IAAI’24/EAAI’24*. AAAI Press.
- Chenyu Zhou, Huacan Chai, Wenteng Chen, Zihan Guo, Rong Shan, Yuanyi Song, Tianyi Xu, Yingxuan Yang, Aofan Yu, Weiming Zhang, Congming Zheng, Jiachen Zhu, Zeyu Zheng, Zhuosheng Zhang, Xingyu Lou, Changwang Zhang, Zhihui Fu, Jun Wang, Weiwen Liu, and 2 others. 2026. *Externalization in llm agents: A unified review of memory, skills, protocols and harness engineering*. *Preprint*, arXiv:2604.08224.

A Training Details

We use Qwen3-8B (Yang et al., 2025a) as the backbone LLM throughout all experiments. During pre-training, skill documents are crawled from GitHub and undergo deduplication and quality filtering. The meta data (name and description) of each skill is stripped, retaining only the markdown body as training input. Training configurations for both stages are described below.

Pretraining. We pretrain the hypernetwork on approximately 171K deduplicated skill documents crawled from GitHub, totaling roughly 300M tokens. Training is conducted on $8 \times \text{H100}$ GPUs for 10 epochs with a batch size of 64, a learning rate of $5e-5$ with 200 warmup steps, and a maximum sequence length of 4,096 tokens. We use AdamW with a weight decay of 0.1.

Supervised Fine-Tuning. We fine-tune the pre-trained hypernetwork on the teacher trajectories released by Xia et al. (2026), comprising 237 complete ALFWorld task trajectories and 500 complete Search-QA task trajectories mixed into a single training set. Training is conducted on $8 \times \text{H100}$ GPUs for 10 epochs with a batch size of 32, a

learning rate of $1e-5$ with 400 warmup steps, and a maximum sequence length of 4,096 tokens. We use AdamW with a weight decay of 0.1.

B Skill Configuration

We adopt the skill library released by Xia et al. (2026). For ALFWorld, skills are matched to tasks by category without retrieval, as shown in Table 5. Note that the Pick and Pick2 task types share the same skill document. For Search-QA, three skill documents correspond to three reasoning types and are matched according to dataset, as shown in Table 6.

Table 5: ALFWorld skill-to-task matching rules.

Task Type	Skill
Pick	Pick And Place Skill
Pick2	Pick And Place Skill
Look	Look At Obj In Light Skill
Clean	Clean Skill
Heat	Heat Skill
Cool	Cool Skill

Table 6: Search-QA skill-to-task matching rules.

Dataset	Skill
NQ	direct_retrieval
TriviaQA	direct_retrieval
PopQA	direct_retrieval
Bamboogle	multi_hop_reasoning
MuSiQue	multi_hop_reasoning
HotpotQA	multi_hop_reasoning comparison
2WikiMultihopQA	multi_hop_reasoning comparison

C Evaluation Details

ALFWorld. We evaluate on both the seen split (140 episodes) and the unseen split (134 episodes). Each episode is capped at 50 steps. Skills are matched to tasks by category without retrieval. We report success rate (%).

Search-QA. We randomly sample 500 examples from each dataset, except Bamboogle which is evaluated on its full set of 125 examples. The retriever is E5 with top- $k=3$ retrieved passages, and each query is allowed a maximum of 4 retrieval steps. Skills are matched according to dataset and annotated question sub-type (see Table 6). We report Exact Match (EM).

D Sub-module Ablation

To localize where skill knowledge resides within the model architecture, we analyze the discriminability of each of the 7 LoRA injection positions in Qwen3-8B (attn_q/k/v/o and mlp_gate/up/down) by computing the gap between within-domain and cross-domain cosine similarity for each position. We then design ablation experiments on the ALFWorld seen and unseen splits to compare six injection configurations: **full** (all 36 layers, all 7 positions), **full:o+d** (all 36 layers, attn_o and mlp_down only), **last6** (layers 30–35, all 7 positions), **last6:o+d** (layers 30–35, attn_o and mlp_down only), **first30** (layers 0–29, all 7 positions), and **first30:o+d** (layers 0–29, attn_o and mlp_down only). All configurations use $\alpha=1$.

As shown in Figure 5, attn_o and mlp_down exhibit substantially higher discriminability gaps than the remaining five positions (pretrain: 0.056/0.105; SFT: 0.050/0.094), while attn_q/k/v show gaps close to zero. This indicates that the parametric differences between skills are concentrated in the information output and integration stages rather than the feature selection stages. After SFT, the gaps at all positions narrow slightly, consistent with the domain-level distance contraction observed in the clustering analysis (§4.3).

The position ablation in Table 7 validates this finding from a performance perspective. The full:o+d configuration uses only 2 out of 7 injection positions yet retains 93.3% of the full configuration’s performance on the seen split (59.3 vs. 63.6). On the unseen split, full:o+d surpasses full by 2.2 percentage points (63.4 vs. 61.2), suggesting that the remaining 5 positions contribute primarily noise that hinders generalization. These two analyses jointly demonstrate that targeted injection into attn_o and mlp_down serves both as a functional validation of the similarity-based finding and as a more favorable deployment strategy that achieves comparable or improved performance with 2/7 of the injection parameters.

E Low-Rank Encoding Analysis

To analyze the intrinsic structure of the LoRA weights generated by the hypernetwork, we compute the Frobenius norm, stable rank, and cumulative top- k singular value energy ratio of the weight increment ΔW for all 5 ALFWorld and 3 Search skills at both the pretrain and SFT stages, assessing the compactness of the hypernetwork’s encoding

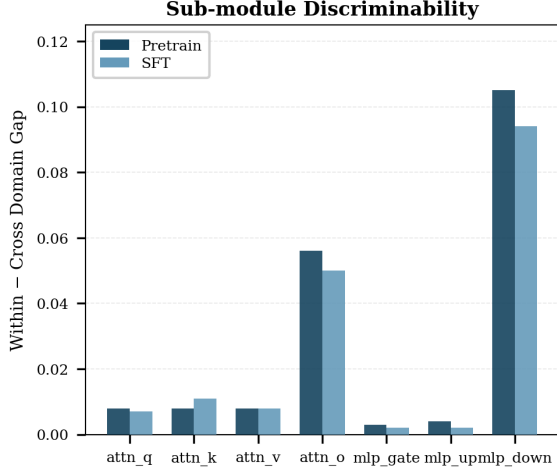


Figure 5: Per-module discriminability (within-domain minus cross-domain cosine similarity gap) for the 7 LoRA injection positions in Qwen3-8B, measured before (Pretrain) and after (SFT) instruction fine-tuning. `attn_o` and `mlp_down` exhibit substantially higher gaps, identifying them as the primary carriers of skill-specific knowledge.

and the effect of training on its structure.

As shown in Table 8, the $\|\Delta W\|$ values of all 8 skills are highly consistent across both stages with no significant cross-domain variation, indicating that the hypernetwork produces weight outputs of stable magnitude regardless of skill text complexity. The stable rank of all skill LoRAs ranges from approximately 2.35–2.40 (pretrain) to 2.17–2.23 (SFT), while a randomly initialized LoRA of the same shape yields a stable rank of 837.87, a difference of roughly $380\times$. As shown in Table 9, the top 2 singular directions alone capture approximately 67% of the total energy, and the top 5 directions capture approximately 93%, confirming that skill knowledge is compressed into a very small number of singular directions and that the hypernetwork achieves genuinely low-rank encoding.

SFT further intensifies this compression. After fine-tuning, the stable rank of all skills decreases uniformly by approximately 0.17. Rank-1 energy increases by roughly 5.3 percentage points for ALFWorld and 5.1 for Search; Rank-2 energy increases by approximately 3.5 and 3.3 percentage points, respectively. The direction and magnitude of these shifts are consistent across both domains, indicating that SFT systematically concentrates skill knowledge into fewer singular directions, improving encoding efficiency.

Table 7: ALFWorld performance (%) under six LoRA injection configurations with $\alpha=1$. The best result per task within each split is highlighted in **bold**. Rows shaded in blue use `attn_o` and `mlp_down` only.

Config	Pick	Look	Clean	Heat	Cool	Pick2	Avg
<i>Seen split</i>							
full	97.1	61.5	66.7	37.5	44.0	50.0	63.6
full:o+d	85.7	61.5	51.9	43.8	48.0	50.0	59.3
last6	88.6	30.8	37.0	25.0	36.0	33.3	47.1
last6:o+d	91.4	53.9	33.3	37.5	24.0	29.2	47.9
first30	91.4	69.2	55.6	37.5	44.0	54.2	61.4
first30:o+d	82.9	76.9	44.4	37.5	56.0	50.0	59.3
<i>Unseen split</i>							
full	75.0	72.2	71.0	43.5	38.1	64.7	61.2
full:o+d	70.8	55.6	64.5	56.5	81.0	47.1	63.4
last6	58.3	55.6	41.9	47.8	71.4	23.5	50.0
last6:o+d	62.5	50.0	45.2	47.8	57.1	17.7	47.8
first30	75.0	66.7	71.0	60.9	33.3	64.7	62.7
first30:o+d	83.3	61.1	58.1	65.2	71.4	17.7	61.2

F Out-of-Distribution Skill Sources

To verify whether the domain-level encoding capability of the hypernetwork generalizes to unseen domains (§4.3), we collect skill texts from public GitHub repositories across three out-of-distribution domains: Code (18 skills), Finance (13 skills), and Writing (11 skills). Table 10 lists the source repositories for each domain.

G Injection Coefficient Analysis

Table 11 reports the full per-task ALFWorld success rates under each LoRA injection coefficient α on both the seen and unseen splits.

H Case Studies for Skill Composition

This appendix provides detailed per-episode analysis of the three merging strategies discussed in §4.5. Each case study examines a specific episode where the merging strategies diverge in outcome, revealing the underlying mechanism of success or failure.

Case 1: Complementary Capability Transfer.

The task in ep4 is to examine a CD under a desklamp. The room contains 5 shelves (shelf 1–5), 3 drawers, and several sidetables; the desk lamp is on sidetable 2 and the CD is on shelf 3. Look-Only fails within 50 steps, never visiting any shelf; its search is confined to the desk, drawers 1–3, sidetables 1–2, safe, and bed, repeatedly cycling through

Table 8: Frobenius norm ($\|\Delta W\| \times 10^{-3}$) and stable rank (SR) of hypernetwork-generated ΔW at Pretrain and SFT stages. SFT columns are shaded in blue.

Skill	$\ \Delta W\ (\times 10^{-3})$		Stable Rank	
	Pretrain	SFT	Pretrain	SFT
<i>ALFWorld</i>				
Clean	2.788	2.843	2.36	2.18
Cool	2.788	2.841	2.36	2.18
Heat	2.785	2.841	2.35	2.17
Look	2.787	2.841	2.36	2.18
Pick	2.787	2.841	2.35	2.17
<i>Search</i>				
Direct	2.790	2.848	2.40	2.23
MultiHop	2.793	2.849	2.39	2.23
Compare	2.790	2.846	2.40	2.23

Table 9: Cumulative singular value energy ratio (%) of top- k directions at Pretrain and SFT stages. SFT columns are shaded in blue.

Skill	Rank-1 (%)		Rank-2 (%)		Rank-5 (%)	
	Pre.	SFT	Pre.	SFT	Pre.	SFT
<i>ALFWorld</i>						
Clean	48.8	54.1	67.1	70.6	92.9	93.7
Cool	48.8	54.2	67.0	70.7	92.9	93.7
Heat	49.1	54.3	67.3	70.8	93.0	93.8
Look	48.9	54.2	67.1	70.7	92.8	93.7
Pick	49.2	54.4	67.4	70.9	93.0	93.8
<i>Search</i>						
Direct	48.0	53.2	66.4	69.7	92.5	93.1
MultiHop	48.2	53.1	66.5	69.7	92.6	93.1
Compare	48.1	53.2	66.4	69.7	92.6	93.1

previously checked locations (desk visited 8 times, drawer 2 opened 3 times). The root cause is that the Look skill lacks a systematic object search strategy: the model successfully locates and activates the desk lamp (Step 12) but cannot find the CD. Component Merging completes the task in 17 steps: after exhausting drawers 1–3, the model begins scanning shelf 1→2→3 at Step 12, discovers the CD on shelf 3 at Step 14, picks it up at Step 15, returns to sidetable 2 at Step 16, and activates the desk lamp at Step 17. The behavioral difference is concentrated in a single dimension: the pick-specific component contributes a systematic search strategy that includes shelves in the search space, a strategy absent from the Look skill’s task-specific component. Because Component Merging mounts the general and mistakes components only once and superposes look-specific and pick-specific independently, Pick’s search capability is introduced

Table 10: Out-of-distribution skill sources collected from public GitHub repositories across three unseen domains.

Domain	GitHub Repository
Code	addyoamani/agent-skills
	jaketestowac/awesome-copilot-for-testers
	github/awesome-copilot
Finance	deanpeters/Product-Manager-Skills
	CaseMark/skills
	borghei/claude-skills
	anthropics/knowledge-work-plugins
Writing	affaan-m/everything-claude-code
	danielabar/meblog
	OpenClaudia/openclaudia-skills
	continuedev/continue
	lguz/humanize-writing-skill
	jpeggdev/humanize-writing
	labarba/sciwrite

cleanly without interfering with Look’s lamp interaction behavior.

Case 2: Weight Redundancy and Decision Disruption.

The task in ep6 is to examine a keychain under a desk lamp. Dresser 1 holds both desk lamp 1 and keychain 3; drawer 7 holds keychain 4; both keychains are valid targets. Look-Only completes the task in 5 steps by simultaneously discovering the desk lamp and keychain on the dresser. Component Merging completes it in 18 steps after systematically scanning drawers 1–7. Direct Merging fails within 50 steps: the model identifies valid targets twice (keychain 4 at Step 27, keychain 3 at Step 35) yet executes no pick-up action after either identification, instead wandering to other locations; the desk lamp is activated 3 times, but the keychain is never in hand when the lamp is on. This pattern reveals the core problem of Direct Merging: the Look and Pick skill texts share identical general and mistakes components, so mounting their complete LoRAs superimposes these shared components twice, over-amplifying general behavioral patterns that persistently suppress task-specific pick-up actions. Direct Merging does not destroy the model’s ability to perceive targets but disrupts the decision threshold for triggering a pick-up upon target recognition. Component Merging avoids this by mounting general and mistakes components once each, with pick-specific supplementing capability only at the task-specific level, keeping the weight strength of general behavioral patterns unchanged and stabilizing the perception-to-action decision chain.

Table 11: ALFWorld success rate (%) under varying LoRA scaling factor α on seen and unseen splits. $\alpha=0$ corresponds to the backbone without LoRA and $\alpha=1$ corresponds to the unscaled hypernetwork output. The best result per task is highlighted in **bold**, and the best average row is shaded in blue.

Seen							
α	Pick	Look	Clean	Heat	Cool	Pick2	Avg
0.0	82.86	46.15	18.52	37.50	32.00	29.17	43.57
0.1	82.86	53.85	33.33	43.75	52.00	37.50	52.86
0.2	82.86	61.54	44.44	50.00	48.00	41.67	56.43
0.3	91.43	69.23	51.85	56.25	48.00	50.00	62.86
0.5	100.0	92.31	48.15	43.75	56.00	62.50	68.57
0.6	97.14	92.31	62.96	43.75	64.00	75.00	74.29
0.8	97.14	76.92	70.37	31.25	52.00	70.83	70.00
1.0	97.14	61.54	66.67	37.50	44.00	50.00	63.57
1.2	65.71	7.69	7.41	6.25	8.00	12.50	22.86

Unseen							
α	Pick	Look	Clean	Heat	Cool	Pick2	Avg
0.0	54.17	55.56	41.94	47.83	57.14	23.53	47.01
0.1	79.17	55.56	48.39	43.48	71.43	11.76	52.99
0.2	75.00	55.56	54.84	47.83	66.67	23.53	55.22
0.3	75.00	55.56	58.06	65.22	71.43	47.06	62.69
0.5	79.17	77.78	70.97	56.52	76.19	64.71	70.90
0.6	91.67	66.67	64.52	43.48	80.95	70.59	69.40
0.8	87.50	66.67	77.42	34.78	38.10	88.24	65.67
1.0	75.00	72.22	70.97	43.48	38.10	64.71	61.19
1.2	33.33	0.00	6.45	4.35	0.00	0.00	8.21

Case 3: OOD Encoding and Behavioral Incoherence.

The task in ep36 is to look at an alarmclock under a desk lamp. Desk 1 holds both alarmclock 2 and desk lamp 1, making this a simple co-located task. Look-Only completes it in 4 steps; Component Merging completes it in 3 steps (go to desk $\rightarrow$ use desk lamp $\rightarrow$ take alarmclock), one of the shortest successful paths across all episodes. Text Merging fails within 50 steps with a failure mode fundamentally different from Case 2: the model correctly identifies and picks up alarmclock 2 at Step 2 but puts it back at Step 3; it correctly activates desk lamp 1 at Step 4 but immediately leaves the desk for sidetable 2 at Step 5. The model then cycles among the desk, sidetable 1, sidetable 2, and bed, activating desk lamp 1 four times and desk lamp 2 five times while the alarmclock remains on the desk. Unlike Direct Merging’s failure mode of perceiving targets but being unable to trigger pick-up, Text Merging exhibits a different pathology: it can execute individual correct actions but cannot chain them into a coherent sequence. Pick-up actions are interrupted by lamp activation, which is in turn interrupted by exploration, with the two

skills’ behavioral patterns alternately activating yet never completing a handoff. The root cause is that Text Merging concatenates two skill texts as a single input to the hypernetwork, but the hypernetwork has only seen single skill texts during training; the concatenated input constitutes an out-of-distribution (OOD) input, and the resulting LoRA fails to separate the two skills’ knowledge, causing their behavioral patterns to interfere at the parameter level without any dispatch mechanism. Component Merging, by contrast, decomposes skills at the text level into independent components, encodes each through the hypernetwork separately, and maintains each behavioral pattern as an independent module in parameter space, thereby supporting coherent sequential decision execution.

I Sensitivity Analysis Details

We apply four types of perturbation to the original skill text and regenerate LoRA weights to evaluate performance changes on ALFWorld and SearchQA. The four perturbations target the semantic content, formatting structure, arrangement order, and information density of skill text, respectively, covering the major quality degradation scenarios that skill texts may encounter in practical deployment.

- **Paraphrase.** We use a fixed prompt and fixed temperature with Claude Sonnet to produce semantically equivalent rewrites of each skill text, preserving the original meaning while altering the wording. All rewritten texts are manually verified to ensure semantic consistency.
- **Plaintext.** We convert Markdown bullet-point formatting into plain-text paragraphs, removing all *, #, and _ markup symbols.
- **Reorder.** We randomly shuffle the order of bullet points within each section, using a fixed random seed to ensure reproducibility.
- **Noise.** We insert one logically irrelevant but grammatically fluent sentence at the end of each rule, before the Apply when field.

Full per-task results are reported in Table 12.

Table 12: Full per-task sensitivity results under four perturbation types. ALFWorld reports success rate (%) and Search-QA reports exact match (%). Each perturbation is evaluated with both In-context Skill and **LatentSkill**. The best result per perturbation is highlighted in **bold**, and **LatentSkill** rows are shaded in blue.

Method	ALFWorld							Search-QA							
	Pick	Look	Clean	Heat	Cool	Pick2	Avg	NQ	Triv	Pop	Hotp	2WK	MuS	Bam	Avg
<i>Base</i>															
In-context	85.7	69.2	70.4	31.3	12.0	33.3	52.9	27.2	56.4	33.0	30.2	39.8	7.60	38.4	32.6
LatentSkill	97.1	92.3	63.0	43.8	64.0	75.0	74.3	36.2	57.6	41.0	39.6	32.0	9.80	25.6	35.6
<i>Paraphrase</i>															
In-context	88.6	61.5	51.9	56.3	16.0	20.8	50.7	28.6	58.8	33.4	31.0	38.4	7.60	37.6	33.2
LatentSkill	97.1	84.6	51.9	43.8	56.0	62.5	67.9	29.8	61.6	36.0	34.2	32.0	9.40	36.8	34.0
<i>Plaintext</i>															
In-context	80.0	61.5	66.7	43.8	12.0	25.0	50.0	27.6	56.4	31.8	31.6	34.8	9.20	43.2	32.4
LatentSkill	97.1	76.9	66.7	56.3	64.0	70.8	74.3	32.4	60.0	35.4	30.8	35.4	10.8	40.0	34.4
<i>Reorder</i>															
In-context	82.9	61.5	59.3	31.3	12.0	41.7	50.7	27.6	56.6	30.8	29.8	37.6	10.4	37.6	32.4
LatentSkill	94.3	84.6	51.9	43.8	64.0	66.7	69.3	32.8	59.6	34.8	28.8	34.0	9.80	40.0	33.6
<i>Noise</i>															
In-context	80.0	61.5	66.7	31.3	12.0	20.8	47.9	24.8	57.0	30.2	32.0	33.6	10.4	40.8	31.7
LatentSkill	91.4	69.2	63.0	50.0	68.0	70.8	71.4	30.6	60.8	35.0	32.2	34.0	8.40	39.2	33.7